

CROWD-SOURCED FAQ GENERATION SYSTEM

Internship Team Project — Full Documentation

May 2026 | 3-Day Sprint Edition

ABSTRACT

The Crowd-Sourced FAQ Generation System is an intelligent, collaborative web platform that harnesses the collective knowledge of a user community to automatically generate, curate, and maintain a high-quality Frequently Asked Questions (FAQ) repository. Traditional FAQ systems rely on a static set of manually written questions and answers, which quickly become outdated and fail to reflect the actual concerns of users. This project addresses that gap by building a dynamic, crowd-powered pipeline where questions are submitted by users, semantically clustered using Natural Language Processing (NLP), prioritized through community voting, and answered through a combination of AI-generated drafts and human expert review.

The system is designed as a full-stack web application with a React-based frontend, a Python/Node.js backend REST API, a relational database for structured storage, and an integrated NLP module for semantic similarity detection and duplicate filtering. Users can submit questions, upvote existing ones, contribute answers, and flag incorrect content. A moderation layer ensures quality control before any answer is published to the public FAQ. The final output is a searchable, categorized, and continuously updated knowledge base that grows smarter over time.

Key technical contributions of this project include: (1) a semantic deduplication engine using sentence embeddings to cluster similar questions, (2) a community voting and ranking algorithm to surface the most relevant questions, (3) an AI-assisted answer drafting pipeline using a large language model API, and (4) a human-in-the-loop moderation workflow. This system demonstrates the practical integration of NLP, full-stack web development, and crowd intelligence — making it a strong showcase of modern software engineering and AI application skills.

PROBLEM STATEMENT

Organizations and platforms maintaining knowledge bases face three core challenges:

- Static Content Decay: FAQ pages are written once and rarely updated, leading to outdated, irrelevant information that fails users.
- Scalability: Human-only FAQ curation does not scale with the growth of user queries and product complexity.
- Duplication & Noise: User-submitted questions often repeat or vary slightly, making manual management inefficient.

This project proposes an automated, crowd-sourced pipeline that solves all three challenges simultaneously by making the FAQ a living, community-maintained document powered by AI.

SYSTEM PIPELINE

The complete pipeline flows through six stages. Each stage is independently developed by a sub-team and integrated in the final sprint.

Stage 1 — Question Submission (Frontend)

Users visit the web app and submit questions via a clean, categorized form. Each submission captures: the question text, category tag (auto-suggested), user identifier (anonymous or logged-in), and timestamp.

- Tech: React form, REST API POST /questions
- Owner: Ganeshprabu, Ritzy, Chaitanya Ram

Stage 2 — Deduplication & Clustering (AI/NLP)

Every incoming question is passed through a sentence embedding model (e.g. sentence-transformers / all-MiniLM). Cosine similarity is computed against existing questions. If similarity > 0.85, the question is merged into an existing cluster. Otherwise, it starts a new cluster.

- Tech: Python, sentence-transformers, scikit-learn, FAISS for vector search
- Owner: Tejeswara Reddy, Nekha, Lohit

Stage 3 — Community Voting & Ranking (Backend + Frontend)

Each question cluster is shown to users for upvoting. A priority score is computed using: upvotes, recency weight, and category relevance. Top-ranked questions enter the answer generation queue.

- Tech: REST API (GET/POST /votes), React voting UI
- Owner: Warish, Vineelkrishna

Stage 4 — AI-Assisted Answer Generation (AI/NLP + Backend)

The top-ranked unanswered questions are sent to an LLM API (Claude / GPT) with a structured prompt. The AI generates a draft answer. Community members can also contribute answer text. Both streams are stored separately for the moderator.

- Tech: Anthropic / OpenAI API, Python backend, answer storage model
- Owner: Tejeswara Reddy, Aryan, Abhishek

Stage 5 — Human Moderation & Review (Backend + Frontend)

Moderators (team members acting as admin) see a dashboard with: question, AI draft answer, community answers, and upvote count. They approve, edit, or reject. Approved answers are tagged as verified.

- Tech: Admin React dashboard, moderation API endpoints
- Owner: Harshith, Pursharth

Stage 6 — Publish & Search (Frontend + Backend)

Approved FAQ entries are pushed to the public FAQ page. A search bar with keyword + semantic search lets users find answers instantly. Categories and tags allow browsing. The cycle repeats — new questions flow back to Stage 1.

- Tech: Elasticsearch or simple full-text search, public FAQ React page
- Owner: Ganeshprabu, Vineelkrishna, Lohit

TEAM STRUCTURE & RESPONSIBILITIES

Member	Role	Module
You (Team Lead)	Architect, Integrator, PM	All stages — oversight
Ganeshprabu BO	Frontend Dev	Question submission form, FAQ display page
Mohd Warish	Backend Dev	REST API, database schema, voting endpoints
Tejeswara Reddy	AI/NLP Engineer	Semantic clustering, deduplication engine
Chaitanya Ram S	Frontend Dev	Voting UI, community answer submission
Ritzy Elsa George	Frontend Dev	Moderator dashboard UI
Vineelkrishna K	Backend Dev	Search API, FAQ publish endpoint
Nekha Mariya Paul	AI/NLP Engineer	Sentence embeddings, similarity scoring
Harshith Sai Suraj	Backend + QA	Moderation API, testing
Pursharth Kaushal	QA + Docs	Test cases, documentation, README
Abhishek Kumar	AI + Backend	LLM API integration, answer generation

Aryan Gaur	AI + Backend	Prompt engineering, answer ranking
Lohit Kumar Pureti	Full Stack	Search, deployment, integration support

3-DAY SPRINT PLAN

Given the 3-day deadline, the team must work in parallel on independent modules and integrate on Day 3. Below is the exact day-by-day breakdown.

DAY 1 — Setup, Scaffold, Build (Today)

Morning (0–4 hrs) — YOU as lead must complete this first:

- **Create GitHub repository, add all 12 teammates as collaborators**
- **Define folder structure: /frontend, /backend, /nlp-service, /docs**
- **Share this document + pipeline diagram with the team on WhatsApp/Slack**
- **Assign each person their module (use table above), confirm on group chat**
- **Create a shared Google Sheet: columns = Name | Task | Status | Done by**

Everyone builds simultaneously (4–12 hrs):

- Frontend team: Scaffold React app, build question submission form with categories
 - Simple fields: Question text, Category dropdown, Submit button
 - On submit: POST to /api/questions (mock API first)
- Backend team: Set up Node/Express or FastAPI, create database tables (Questions, Answers, Votes, Users), expose /api/questions and /api/votes endpoints
- AI/NLP team: Write Python script that takes a question text, computes embedding, checks similarity against stored questions, returns cluster ID or 'new'
- QA: Write a 1-page test plan covering each module

DAY 2 — Connect, Integrate, Add AI

Morning:

- Frontend connects to real backend API (replace mocks)
- Backend connects to NLP service via HTTP call
- Voting endpoint live — frontend can upvote questions
- Moderation dashboard built (basic list of unanswered questions + approve button)

Afternoon:

- AI answer generation: when a question is approved for answering, call LLM API, store result
- Moderator can see AI draft answer, edit and approve
- Approved answers appear on public FAQ page

- Search: add basic keyword search to FAQ page

Evening:

- Full pipeline walkthrough: submit question → cluster → vote → AI answer → moderate → publish → search
- Fix broken connections, document any missing endpoints

DAY 3 — Polish, Test, Demo & Commit

Morning:

- QA team runs all test cases, reports bugs to backend/frontend
- Fix critical bugs only (not enhancements)
- Pursharth writes README.md with setup instructions

Afternoon:

- Record a 3–5 minute demo video walking through the pipeline
- Prepare slides (5 slides: Problem, Solution, Pipeline, Demo, Team)
- Final commit with message: 'v1.0 – Crowd-Sourced FAQ System – Team submission'
- Push all code to GitHub, verify all branches merged to main

commit as Team Lead:

- System architecture diagram (in /docs folder)
- This project document as PDF + DOCX (in /docs folder)
- README.md at root
- Integration glue code (any file that calls multiple services together)
- Final merge + release tag on GitHub

TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	React.js + Tailwind CSS	User interface — submission, voting, FAQ display
Backend API	Node.js (Express) or FastAPI	REST endpoints, business logic
Database	PostgreSQL or SQLite (quick)	Store questions, answers, votes
NLP Service	Python + sentence-	Semantic similarity, clustering

	transformers	
AI Answers	Claude API / OpenAI GPT	Draft answer generation
Search	Basic SQL LIKE or MiniSearch.js	Keyword search on FAQ page
Version Control	GitHub	Collaboration and submission
Deployment	Localhost / Render.com (free)	Demo environment

WHAT YOU (TEAM LEAD) SHOULD BUILD

As team lead, your job is not to write all the code — it is to make sure everyone can work and that pieces fit together. Here is your personal task list:

- **GitHub repo setup with folder structure and branch rules**
 - **API contract document: define all endpoint URLs, request/response formats, share with team on Day 1 morning**
 - **Integration file: a single script or page that calls all services in sequence to test the full pipeline**
 - **Pipeline diagram (already done — include it in /docs)**
 - **Project document (this document — include it in /docs)**
 - **Daily standup messages in group chat: 'What did you finish? What are you doing next? Any blockers?'**
 - **Final merge: collect all pull requests and merge them into main on Day 3**
 - **Demo script: write a 10-step walkthrough you will narrate during the demo**
-

GITHUB — HOW TO COMMIT & SHOW YOUR WORK

Follow these exact commands to set up and commit your work:

Initial Setup (You do this on Day 1 morning)

```
git init
git remote add origin https://github.com/YOUR_USERNAME/crowd-faq-system.git
mkdir frontend backend nlp-service docs touch README.md docs/architecture.md
git add .
git commit -m 'Initial project structure'
git push -u origin main
```

Daily Commit Pattern (Everyone follows this)

- Morning commit: 'feat: [your-name] scaffold [module-name]'
- Afternoon commit: 'feat: [your-name] connect [module] to API'
- Evening commit: 'fix: [your-name] resolve [bug description]'

Final Submission Commit (Day 3)

```
git add . git commit -m 'v1.0 - Crowd-Sourced FAQ System - Final Submission' git tag v1.0  
git push origin main --tags
```

MINIMUM VIABLE PRODUCT (MVP) FOR 3 DAYS

If some modules are not complete, the MVP you MUST have working for the demo is:

- **A form where someone can type and submit a question**
- **A list of submitted questions with an upvote button**
- **A button that sends the top question to an AI and shows the generated answer**
- **An admin page where a moderator clicks 'Approve' to publish the answer**
- **A public FAQ page that shows approved Q&A pairs with a search bar**

This 5-step flow IS the project. Everything else (clustering, embeddings, advanced search) is an enhancement. Get the MVP working first, then add enhancements.

SHORT ABSTRACT (For Submission Form / Report Cover)

This project presents a Crowd-Sourced FAQ Generation System — a full-stack web application that enables users to submit questions, vote on their priority, and collaboratively build a curated knowledge base. The system integrates NLP-based semantic clustering to deduplicate questions, a community voting mechanism to surface the most relevant queries, and an AI-powered answer generation pipeline using large language models. A human moderation layer ensures answer quality before publication. The result is a dynamic, self-improving FAQ system that scales with the community it serves.

Word count: ~95 words. Use this for any abstract field in your submission portal.